

CPT204 Final Exam Sample

Part A. Gap-fill Questions

1. _____ allows different classes to define methods with the same name differently, while _____ ensures that internal class data is shared by all instances of the class.

2. _____ store objects that are processed in a last-in, first-out fashion, while _____ are processed in a first-in, first-out fashion.

3. Given the following method call:

```
public static void update(int num, int[] array) {  
    num = 100;  
    array[0] = 50;  
}
```

What is the printed value of x in the main method?

```
int x = 10;  
int[] arr = new int[1];  
update(x, arr);  
System.out.println("x = " + x);
```

4. What is the default value of myArray[2] after this declaration?

```
boolean[] myArray = new boolean[5];
```

5. What is the output of the following code:

```
String s1 = "Hello";  
String s2 = new String("Hello");
```

```
System.out.println(s1 == s2);
```

6. Given the Student class below:

```
public class Student {  
    private List<String> courses;  
  
    public List<String> getCourses() {  
        return courses;  
    }  
}
```

Although `courses` is private, the class is still not immutable because the getter returns a/an _____ to a mutable object.

```
7. public boolean equals(Object o) {  
    if (o instanceof Circle) {  
        return radius == ((Circle) o).radius;  
    }  
    return false;  
}
```

After _____, you can access the field. Equality must be based on comparing relevant fields.

```
8. class B {  
    public void p(double i){  
        System.out.println(i *2);  
    }  
}  
  
class A extends B {
```

```
public void p(int i) {  
    System.out.println(i);  
}
```

Given that method overloading is based on the reference by:

```
B a = new A();  
a.p(10);
```

9. In the code below:

```
interface T1 {  
    int K = 1;  
}  
  
System.out.println(T1.K);
```

What kind of static member — which is fixed and cannot be changed — is accessed using the interface name T1?

10. `House house1 = new House(1, 1750.50);`

`House house2 = (House)(house1.clone());`

`System.out.println(house1.getWhenBuilt() == house2.getWhenBuilt());`

The code above performs a/an _____ if `clone()` is not overridden.

11. If the key is the first element in the array, the linear search completes in _____ comparisons.

12. In our class, we introduced various methods for recording OOP learning, among which a _____ is a means of recording ideas, personal thoughts and experiences, as well as insights you gain in the learning process.

13. We have learned several different data structures. Which one is a container that can support efficient and random access and supports insertion and deletion from the end. The best data structure for the container is _____.

14. We have learned several different data structures. Which one is a container that can support efficient and random access and supports insertion and deletion from the end by multiple threads. The best structure for the container is _____.

15. What is the output of the following code segment

```
PriorityQueue<String> queue = new PriorityQueue<String>(  
Arrays.asList("Liverpool", "Suzhou", "London", "Beijing"));  
while (!queue.isEmpty())  
System.out.print(queue.poll() + " ");
```

16.

Suppose we are given a linked list CPTList and list node CPTListNode outlined below.

```
public class CPTListNode {  
    int data;  
    CPTListNode next;  
}
```

```
public class CPTList {  
    private CPTListNode front;  
    // some methods  
}
```

A method called "method1" for the CPTList class is shown below. Assume there are two CPTList objects, list1 and list2. list1 contains 3 nodes of value 1, 2, and 3 (starting from the front). list2 contains another 3 nodes of value 4, 5, and 6 (starting from the front). What would be the

nodes in list1 after list1.method1(list2)? Place a comma in your answer, e.g. 1,3,2,4 (although comma is not a part of the answer)

```
public void method1(CPTList other){  
    CPTListNode l = other.front;  
    while (l.next != null){  
        l = l.next;  
    }  
    l.next = front;  
    front = other.front;  
}
```

17. A Map is a special type of Java container that stores a collection of key/value pairs. We have learned 3 types of built-in maps. They are: ____, ____, and TreeMap.

18. Suppose we have 3 containers below each containing the same set of students IDs at xjtl. container1 is a LinkedHashSet.

container2 is a LinkedList.

container3 is an ArrayList.

The student IDs are stored in a random order in all 3 containers. Now We want to check if a given id, say 2209999, exists in the containers using the method "contains()". The code segment would be like

```
container1.contains(2209999)
```

```
container2.contains(2209999)
```

```
container3.contains(2209999)
```

Which container is likely to have the minimum time consumption?

19. In Big O notation, the term ____ is used to describe algorithms with complexity that grow at a linear rate with the input size.

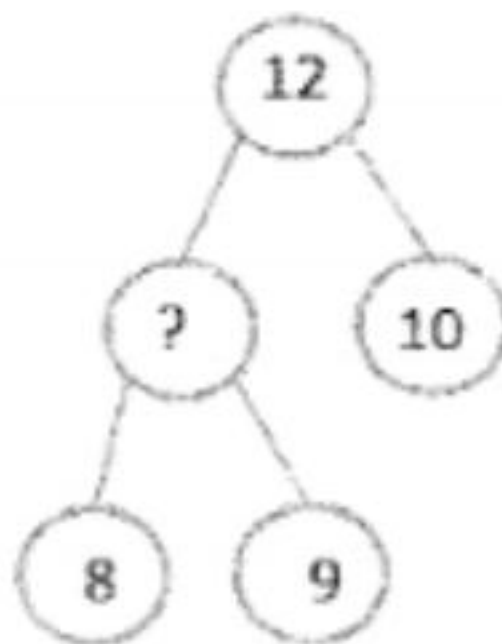
20. The worst-case time complexity for selection sort algorithm is _____.

```
21. int count = 0;
    for (int i = 1; i < n; i += 2) {
        count++;
    }
```

What is the time complexity of the code above?

22. In Merge Sort, the array is repeatedly divided into smaller sublists until each sublist contains only _____ element.

23. What value should best replace the '?' to maintain the max heap strictly?



24. The time complexity for inserting an element into, and deleting an element from, a binary search tree are _____ and _____ respectively, assuming that the tree is balanced.

25. What does Dijkstra's algorithm compute?

26. In an AVL tree, a _____ imbalance occurs when a node is inserted into the right subtree of the right child of A, while a _____ imbalance occurs when a node is inserted into the left subtree of the left child of A.

27. The postorder traversal of the elements in an AVL tree in this order is _____.

28. The following code uses _____ probing to resolve collisions.

```
int index = key % table.length;
while (table[index] != null) {
    index = (index + 1) % table.length; // linear probing
}
table[index] = key;
```

29. What EDI principle is demonstrated by using accessible color schemes in a Java Swing GUI, ensuring that users with visual impairments are not excluded?

30. What EDI principle is reflected when a Java app project is designed to accommodate both younger and older users, screen reader users, and speakers of different languages?

Part B. Coding Questions

Question 1

Scorable Quiz System

Given the following interface:

```
public interface Scorable {
    int getScore();
}
```

Complete the class QuizResult that implements Scorable. The getScore() method should return the percentage score as an integer (e.g., 80 for 8/10). If there are no questions, simply return 0. Use standard rounding (i.e., round to the nearest whole number).

Starter Code:

```
public class QuizResult implements Scorable {  
    private int correctAnswers;  
    private int totalQuestions;  
  
    public QuizResult(int correctAnswers, int totalQuestions) {  
        // TO DO  
    }  
  
    // TO DO  
}
```

Question 2

You are writing a Java program that implements an airplane ticket. The ticket can be compared by passenger name, ticket price, arrival time, and seat class in that order.

Moreover, the seat class is implemented using an enumeration with First Class, Business, and Economy, compared in that order.

Your task is to complete the TicketComparator class.

Date and Comparator have been imported. You are not allowed to import other classes.

The skeleton code is given below:

```
import java.util.Date;  
import java.util.Comparator;  
  
public class AirplaneTicket {  
    private String passengerName;  
    private int ticketPrice;  
    private String flightNumber;  
    private Date arrivalTime;
```



```
private SeatClass seatClass;

public enum SeatClass {
    FIRST_CLASS, BUSINESS, ECONOMY
}

public AirplaneTicket(String passengerName, int ticketPrice,
arrivalTime, SeatClass seatClass) {
    this.passengerName = passengerName;
    this.ticketPrice = ticketPrice;
    this.flightNumber = flightNumber;
    this.arrivalTime = arrivalTime;
    this.seatClass = seatClass;
}

public String getPassengerName() {
    return passengerName;
}

public int getTicketPrice() {
    return ticketPrice;
}

public String getFlightNumber() {
    return flightNumber;
}

public Date getArrivalTime() {
    return arrivalTime;
}

public SeatClass getSeatClass() {
```

```
        return seatClass;
    }

    public static Comparator<AirplaneTicket> getTicketComparator() {
        return new TicketComparator();
    }

    // Complete the class TicketComparator
    private static class TicketComparator implements Comparator<AirplaneTicket> {

    }
}
}
```

----- END OF THE CPT 204 FINAL EXAM SAMPLE QUESTIONS -----